

Exp-1: Red Black-

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node{
```

```
    int data;
```

```
    char color;
```

```
    struct Node *left,*right,*parent;
```

```
};
```

```
struct Node* root=NULL;
```

```
struct Node* newNode(int data){
```

```
    struct Node* n=(struct Node*)malloc(sizeof(struct Node));
```

```
    n->data=data;
```

```
    n->color='R';
```

```
    n->left=n->right=n->parent=NULL;
```

```
    return n;
```

```
}
```

```
void leftRotate(struct Node* x){
```

```
    struct Node* y=x->right;
```

```
    x->right=y->left;
```

```
    if(y->left) y->left->parent=x;
```

```

y->parent=x->parent;
if(!x->parent) root=y;
else if(x==x->parent->left) x->parent->left=y;
else x->parent->right=y;
y->left=x;
x->parent=y;
}

```

```

void rightRotate(struct Node* x){
    struct Node* y=x->left;
    x->left=y->right;
    if(y->right) y->right->parent=x;
    y->parent=x->parent;
    if(!x->parent) root=y;
    else if(x==x->parent->left) x->parent->left=y;
    else x->parent->right=y;
    y->right=x;
    x->parent=y;
}

```

```

void fixInsert(struct Node* pt){
    while(pt != root && pt->parent && pt->parent->color == 'R'){
        struct Node* v = pt->parent;
        if(!v->parent) break;
        struct Node* u = v->parent;
        if(v == u->left){
            struct Node* w = u->right;
            if(w && w->color == 'R'){

```

```

    v->color = 'B';

    w->color = 'B';

    u->color = 'R';

    pt = u;
}

else{

    if(pt == v->right){

        pt = v;

        leftRotate(pt);

    }

    if(pt->parent){

        pt->parent->color = 'B';

        if(pt->parent->parent){

            pt->parent->parent->color = 'R';

            rightRotate(pt->parent->parent);

        }

    }

}

}

else{

    struct Node* w = u->left;

    if(w && w->color == 'R'){

        v->color = 'B';

        w->color = 'B';

        u->color = 'R';

        pt = u;

    }

    else{

```

```

        if(pt == v->left){
            pt = v;
            rightRotate(pt);
        }
        if(pt->parent){
            pt->parent->color = 'B';
            if(pt->parent->parent){
                pt->parent->parent->color = 'R';
                leftRotate(pt->parent->parent);
            }
        }
    }
}

if(root) root->color = 'B';
}

void insert(int data){
    struct Node* n=newNode(data);
    struct Node *y=NULL,*x=root;
    while(x){
        y=x;
        if(n->data<x->data) x=x->left;
        else x=x->right;
    }
    n->parent=y;
    if(!y) root=n;
    else if(n->data<y->data) y->left=n;
    else y->right=n;
}

```

```

        fixInsert(n);
    }
void levelOrder(){
    if(!root) return;
    struct Node* q[1000];
    int f=0,r=0;
    q[r++]=root;
    while(f<r){
        int c=r-f;
        while(c--){
            struct Node* t=q[f++];

            printf("%d(%c) ",t->data,t->color);

            if(t->left) q[r++]=t->left;
            if(t->right) q[r++]=t->right;
        }
        printf("\n");
    }
}
int main(){
    int n,x;
    char op;
    scanf("%d",&n);
    while(n--){
        scanf(" %c %d",&op,&x);
        if(op=='I') insert(x);
    }
    levelOrder();
}

```

```
    return 0;
}
```

Exp-2:

Fractional Knapsack

```
#include <stdio.h>

struct Item {
    int value;
    int weight;
    float ratio;
};

int main() {
    int n;
    float W;
    scanf("%d", &n);
    scanf("%f", &W);
    struct Item items[n];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &items[i].value, &items[i].weight);
        items[i].ratio = (float)items[i].value / items[i].weight;
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (items[i].ratio < items[j].ratio) {
                struct Item temp = items[i];
                items[i] = items[j];
                items[j] = temp;
            }
        }
    }
}
```

```

    }
}

float maxVal = 0.0;
for (int i = 0; i < n && W > 0; i++) {
    if (items[i].weight <= W) {
        maxVal += items[i].value;
        W -= items[i].weight;
    } else {
        maxVal += items[i].ratio * W;
        W = 0;
    }
}

printf("%.2f\n", maxVal);
return 0;
}

```

Exp 3-

Job sequencing

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {
```

```
    int id;
```

```
    int deadline;
```

```
    int profit;
```

```
} Job;
```

```
int compare(const void *a, const void *b) {
```

```
    Job *jobA = (Job *)a;
```

```
    Job *jobB = (Job *)b;
```

```

        return jobB->profit - jobA->profit;
    }

int main() {
    int n;

    scanf("%d", &n);

    Job jobs[n];

    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &jobs[i].id, &jobs[i].deadline, &jobs[i].profit);
    }

    qsort(jobs, n, sizeof(Job), compare);

    int maxDeadline = 0;

    for (int i = 0; i < n; i++) {
        if (jobs[i].deadline > maxDeadline) {
            maxDeadline = jobs[i].deadline;
        }
    }

    int slot[maxDeadline];

    for (int i = 0; i < maxDeadline; i++) {
        slot[i] = -1;
    }

    int totalProfit = 0;

    for (int i = 0; i < n; i++) {
        for (int j = jobs[i].deadline - 1; j >= 0; j--) {
            if (slot[j] == -1) {
                slot[j] = jobs[i].id;
                totalProfit += jobs[i].profit;
                break;
            }
        }
    }
}

```



```

    }
}

printf("Scheduled tasks: ");
for (int i = 0; i < maxDeadline; i++) {
    if (slot[i] != -1) {
        printf("%d ", slot[i]);
    }
}

printf("\nMaximum Profit: %d\n", totalProfit);

return 0;
}

```

Exp-4:

Currency exchange

```
#include<stdio.h>
```

```
#include<limits.h>
```

```
int main(){
```

```
    int n;
```

```
    scanf("%d",&n);
```

```
    int coins[n];
```

```
    for(int i=0; i<n; i++){
```

```
        scanf("%d",&coins[i]);
```

```
    }
```

```
    int t;
```

```
    scanf("%d",&t);
```

```
    int dp[t+1];
```

```
    dp[0]=0;
```

```
    for(int i=1;i<=t;i++){
```

```

        dp[i]=INT_MAX;
    }
    for(int i=0;i<=t;i++){
        for(int j=0;j<n;j++){
            if(coins[j]<=i && dp[i-coins[j]]!= INT_MAX){
                if(dp[i-coins[j]]+1 < dp[i]){
                    dp[i]=dp[i-coins[j]]+1;
                }
            }
        }
    }
    if(dp[t]==INT_MAX){
        printf("-1\n");
    }else{
        printf("Minimum coins required: %d\n",dp[t]);
    }
    return 0;
}

```

Exp 5-

Matrix chain multiplication

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAXN 105
```

```
long long dp[MAXN][MAXN];
```

```
int main(){
```

```
    int n;
```

```
    scanf("%d",&n);
```

```

int p[MAXN];
for(int i=0;i<=n;i++){
    scanf("%d",&p[i]);
}
for(int len=2;len<=n;len++){
    for(int i=1;i<=n-len+1;i++){
        int j=i+len-1;
        dp[i][j]=LLONG_MAX;
        for(int k=i;k<j;k++){
            long long cost=dp[i][k]+dp[k+1][j]+(long long)p[i-1]*p[k]*p[j];
            if(cost<dp[i][j]){
                dp[i][j]=cost;
            }
        }
    }
}
printf("%lld\n",dp[1][n]);
return 0;
}

```

Exp 6-

Rabin Karp

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define d 256
```

```
#define q 101
```

```
void rabinKarp(char text[], char pattern[]){
```

```
    int n = strlen(text);
```

```

int m = strlen(pattern);

int i, j;

int p = 0;

int t = 0;

int h = 1;

int found = 0;

/* Calculate h = (d^(m-1)) % q */
for (i = 0; i < m - 1; i++){
    h = (h * d) % q;
}

/* Calculate initial hash values */
for (i = 0; i < m; i++){
    p = (d * p + pattern[i]) % q;
    t = (d * t + text[i]) % q;
}

/* Slide the pattern over text */
for (i = 0; i <= n - m; i++){
    if (p == t){
        for (j = 0; j < m; j++){
            if (text[i + j] != pattern[j]){
                break;
            }
        }
        if (j == m){
            printf("Pattern found at index: %d\n", i);
            found = 1;
        }
    }
}

```

```

        if (i < n - m){
            t = (d * (t - text[i] * h) + text[i + m]) % q;

            if (t < 0){
                t = t + q;
            }
        }
    }
    if (found == 0){
        printf("Pattern not found\n");
    }
}

int main(){
    char text[1000];
    char pattern[100];
    scanf("%s", text);
    scanf("%s", pattern);
    rabinKarp(text, pattern);
    return 0;
}

```

Exp 7

-LCS

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char X[1001], Y[1001];
```

```
    scanf("%s %s", X, Y);
```

```

int m = strlen(X), n = strlen(Y);

int dp[1001][1001];

for (int i = 0; i <= m; i++) {
    for (int j = 0; j <= n; j++) {
        if (i == 0 || j == 0)
            dp[i][j] = 0;
        else if (X[i-1] == Y[j-1])
            dp[i][j] = dp[i-1][j-1] + 1;
        else
            dp[i][j] = dp[i-1][j] > dp[i][j-1] ? dp[i-1][j] : dp[i][j-1];
    }
}

int len = dp[m][n];

char lcs[1001];

int idx = len;

lcs[idx] = '\0';

int i = m, j = n;

while (i > 0 && j > 0) {
    if (X[i-1] == Y[j-1]) {
        lcs[--idx] = X[i-1];
        i--; j--;
    } else if (dp[i-1][j] >= dp[i][j-1]) {
        i--;
    } else {
        j--;
    }
}

printf("Length of LCS: %d\n", len);

```

```
    printf("LCS: %s\n", lcs);  
    return 0;  
}
```

Exp 8-

CEA

```
#include <stdio.h>
```

```
// Recursive approach
```

```
long long fibRecursive(int n) {  
    if (n <= 1)  
        return n;  
    return fibRecursive(n - 1) + fibRecursive(n - 2);  
}
```

```
// Non-recursive (Dynamic Programming / Iterative) approach
```

```
long long fibNonRecursive(int n) {  
    if (n <= 1)  
        return n;  
    long long a = 0, b = 1, c;  
    for (int i = 2; i <= n; i++) {  
        c = a + b;  
        a = b;  
        b = c;  
    }  
    return b;  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```
printf("Recursive Fibonacci(%d): %lld\n", n, fibRecursive(n));  
printf("Non-Recursive Fibonacci(%d): %lld\n", n, fibNonRecursive(n));  
return 0;  
}
```